Introduction to Deep Learning

Session 10: CNNs in Practice

May 2026

Magda Gregorová - magda.gregorova@thws.de

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

Lecture Overview

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

1. Transfer Learning

2. Practical CNN Training

3. Beyond Classification

Three Regimes

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Regime	When	Data needed	Compute
Train from scratch	Novel architecture, very different domain	Large labelled dataset	Very high
Pretrain + fine-tune	Moderate dataset, research setting	Moderate	Medium
Download & adapt	Most practical tasks	Small–medium	Low

Modern workflow:

1. download pretrained from torchvision / timm / HuggingFace
2. adapt to task

Training from scratch rare - large organisations with compute and data budget

Why Pretrained Models Work

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Key insight: CNN layers learn features of increasing specificity

Early layers — general:

- Edges, colours, orientations
- Useful across almost all visual tasks
- Transfer well to new domains

Later layers — specific:

- Object parts, textures, patterns
- Specific to source task
- Need adaptation for new task

Pretrained: ImageNet (1.2M images), LAION-5B (5.8B image-text pairs), proprietary

Works because: low-level visual features universal - edges exist in all images

Feature Extraction vs Fine-tuning

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Feature extraction:

- Freeze all pretrained layers
- Replace and train FC head only

\+ Fast, few trainable parameters
\+ Works with small datasets
\+ No risk of catastrophic forgetting
\- Limited adaptation to new domain

Fine-tuning:

- Unfreeze some or all layers
- Train with small learning rate

\+ Better performance
\+ Adapts to new domain
\- Needs more data
\- Risk of catastrophic forgetting

Catastrophic forgetting: large learning rate overwrites pretrained weights — use 10×–100× smaller learning rate than original training

When to Use Which

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

	Similar domain	**Different domain**
Small dataset	Feature extraction (freeze all, train head)	Feature extraction (try different layer depths)
Large dataset	Fine-tune all layers (small learning rate)	Fine-tune all layers (more epochs, careful tuning)

Rule of thumb:

- Less data → freeze more layers
- More similar to source domain → reuse more layers
- Always start with feature extraction, add fine-tuning if needed

Practical Tuning

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Freezing strategy:

- Start fully frozen — train head only
- Unfreeze progressively from top (later layers first)
- Earlier layers = more general = freeze longer

Learning rates:

- New head: standard learning rate (e.g. 1e-3)
- Fine-tuned layers: much smaller (e.g. 1e-5)
- **Discriminative learning rates** — different rates per layer group
- Warmup — start small, increase gradually → new head stabilization

Other considerations:

- More aggressive data augmentation when dataset is small
- Early stopping essential — fine-tuning overfits quickly

Practical CNN Training

torchvision and timm

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Two main sources of pretrained models:

torchvision.models:

- Official PyTorch model zoo
- ResNet, VGG, EfficientNet, ViT
- Pretrained on ImageNet
- Simple API

```
from torchvision.models import resnet50,
  ResNet50_Weights
model = resnet50(weights=ResNet50_Weights.DEFAULT)
```

timm (PyTorch Image Models):

- 700+ pretrained models
- State-of-the-art architectures
- Multiple pretraining datasets
- Community standard

```
import timm
model = timm.create_model('resnet50',
  pretrained=True)
```

HuggingFace: `transformers` library for vision-language models (CLIP, ViT, DINO)

Adapting Pretrained Model

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Replace classifier head:

```
model = resnet50(weights=ResNet50_Weights.DEFAULT)

# freeze all layers
for param in model.parameters():
   param.requires_grad = False

# replace FC head
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, num_classes)

# only head parameters are trainable
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

For fine-tuning: unfreeze layers selectively

```
# unfreeze last ResNet block
for param in model.layer4.parameters():
   param.requires_grad = True

# fine-tune with discriminative learning rate
optimizer = torch.optim.Adam([
   {'params':  model.layer4.parameters(), 'lr':  1e-5},
   {'params':  model.fc.parameters(), 'lr':  1e-3},
])
```

All basics hold:

```
model.train()

model.eval()

optimizer.zero_grad()
```

Image Transforms Pipeline

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Standard training/ validation / test pipeline:

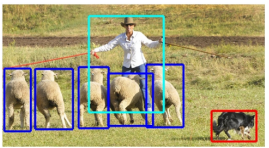
```
from torchvision import transforms

train_transform = transforms.Compose([
   transforms.RandomResizedCrop(224),
   transforms.RandomHorizontalFlip(),
   transforms.ColorJitter(brightness=0.2, contrast=0.2),
   transforms.ToTensor(),
   transforms.Normalize(mean=[0.485, 0.456, 0.406],std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
   transforms.Resize(256),
   transforms.CenterCrop(224),
   transforms.ToTensor(),
   transforms.Normalize(...)
])
```

- No augmentation at validation/ test — deterministic transforms only
- Normalisation statistics must match pretraining - check torchvision / timm

Beyond Classification

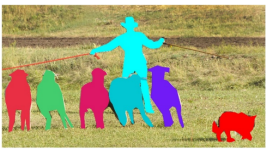
Vision Tasks

Classification



Object detection

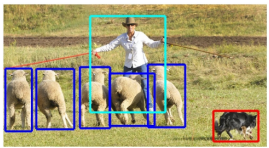
Semantic segmentation



Instance segmentation

All built on CNN backbones — pretrained feature extractors + task-specific heads

Object Detection



Output: bounding boxes + class labels + confidence scores per object instance

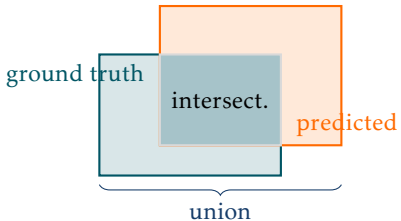
Annotation: bounding box around every object instance

Two main strategies:

- **Two-stage** (Faster R-CNN): region proposal network → classify and refine — higher accuracy, slower
- **One-stage** (YOLO, SSD): directly predict boxes and classes in single pass — faster, real-time capable

Evaluation: IoU and mAP — see next slide

Evaluation — IoU and mAP



IoU (Intersection over Union):

- Overlap between predicted and ground truth box
- Prediction correct if IoU > threshold (e.g. 0.5)
- $\text{IoU} = \frac{|\text{intersection}|}{|\text{union}|} \in [0, 1]$

Average Precision (AP):

- Vary confidence threshold — plot precision-recall curve
- AP = area under that curve per class
- mAP = mean AP across all classes

Semantic Segmentation

Output: pixel-wise class label map — same spatial size as input

Annotation: careful pixel-level boundary tracing

Main strategy: encoder-decoder architecture

- **Encoder:** CNN backbone reduces resolution, extracts features
- **Decoder:** upsamples back to input resolution
- **Skip connections:** recover spatial detail lost in encoding

Key methods: U-Net, DeepLab v3, SegFormer

Evaluation: mean Intersection over Union (mIoU) — mean IoU across all classes

Instance Segmentation

Output: per-instance binary mask + class label — distinguishes individual objects of same class

Annotation: per-instance pixel masks

Main strategy: detect then segment

- Mask R-CNN: extends Faster R-CNN with mask prediction head — binary mask per detected box
- SAM (Segment Anything): foundation model, prompt-based — any object, zero-shot

Panoptic segmentation: unified — semantic for background, instance for objects — modern standard

Evaluation: mAP on masks rather than boxes

Annotation Cost

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Task	Annotation type	Time per image	Example dataset
Classification	Image-level label	Seconds	ImageNet
Detection	Bounding boxes	Minutes	COCO
Semantic segmentation	Pixel-wise labels	30–90 min	Cityscapes
Instance segmentation	Per-instance masks	Hours	COCO

Annotation cost = the real bottleneck — not model complexity

Modern tools to reduce cost:

- SAM (Segment Anything Model) — interactive, prompt-based annotation assistance
- Semi-supervised and weakly supervised methods — learn from cheap labels
- Synthetic data — generate annotated data automatically

Summary

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

Transfer learning:

- Download pretrained model
- Feature extraction vs fine-tuning
- Freeze progressively

Practical tools:

- Pretrained model libraries
- Image transformations

Beyond classification:

- Object detection
- Semantic segmentation
- Instance segmentation
- All use CNN backbones